DEMO USING AGENT MODE **(tested with GPT 5.3 codex)**

Enable "Auto-approve" to avoid many confirmations

Disable SKILL

Selezionare database Bergamo_2026

--- after confirmation for LLM, back to slides ---

PROMPT 1 – INVESTIGATIONS ON QUERY STORE

```
Investigate the SQL performance problem (SQL machine time) in the current database.
Investigate the issue using Query Store.

Automatically identify the dominant performance resource, the main contributing query, its Query ID, all associated execution plans, and its Query Store runtime history.

Investigate whether performance degraded while the same execution plan was still being used.

Then investigate the evidence required to explain the degradation, including query selectivity, data distribution, statistics, and indexes when relevant.

Do not ask me to provide Query IDs or Plan IDs if they can be discovered automatically.

Use only read-only diagnostic queries.

Do not create or drop indexes, update or drop statistics, force plans, clear caches, modify Query Store, or modify data.

Separate the response into:
    1. confirmed evidence
    2. observations
    3. hypotheses
    4. missing evidence
    5. diagnosis
    6. recommended actions.
```

OUTPUT:

1) Confirmed evidence

- **Main contributing query (auto-identified):**
  - `query_id = 1`
  - Object: `[dbo].[GiveNotes]`
  - Text (first 100 chars):
    `(@N2 int,@N1 int)SELECT count(Name),name, 'easyfilter' AS easyfilter FROM Notes where ID BETWEEN @N2`
  - Total executions: `11,517`
  - Total CPU: `1,382,706.82 ms`
- **Associated execution plans (Query Store):** `plan_id` = `1, 6, 11, 14`.
  - Plan CPU contribution:
    - `14`: `1,274,552.27 ms` (~92% of query CPU), avg CPU/execution `755.07 ms`
    - `11`: `67,728.65 ms`
    - `6`: `39,077.24 ms`
    - `1`: `1,348.66 ms`
- **Runtime history shows degradation while same plan stayed active (`plan_id = 14`):**
  - Interval `13:50` → `13:54` UTC:
    - avg CPU: `280.39` → `2,564.88 ms` (**9.15x**)
    - avg logical reads: `4,224` → `34,738` (**8.22x**)
    - avg rowcount: `32.96` → `38.32` (**1.16x**)
- **Dominant wait category on degraded plan (`plan_id = 14`):**
  - `Parallelism` wait = `448,422 ms` (**91.50%** of plan wait time)
- **Selectivity/data distribution evidence (`dbo.Notes`):**
  - Rows: `8,388,606`
  - Distinct `ID`: `44`; distinct `NAME`: `44`
  - `ID` range: `11415` to `98996`
  - Top `ID` value has `4,194,303` rows (~50% of table) → extreme skew
  - Compiled parameter ranges for plans cover effectively **all table rows** (`~99.5%` to `100%` rows in range)
- **Indexes/statistics evidence:**
  - Only index: clustered PK on `id2` (`PK_Notes`)
  - Predicate/group columns are `ID`, `NAME` (not indexed as keys)
  - Auto stats exist on `ID` and `NAME`; sampled only `4.05%`, `10` histogram steps, `modification_counter = 0`

2) Observations

- The issue is concentrated in **one query** and mainly one plan (`plan_id 14`).
- Performance worsened **without plan change** during the degraded window.

- The query shape (`ID BETWEEN ... + GROUP BY NAME`) runs over a table where `ID/NAME` are highly skewed and low-cardinality.
- Wait profile is parallelism-heavy, consistent with expensive parallel processing/scans on large row volumes.

3) Hypotheses

- **Same-plan runtime instability** due to changing runtime row distribution/concurrency, not necessarily recompilation.
- **Low selectivity predicate** on non-key indexed columns drives very large row reads.
- **Skew + limited histogram detail** (low sample, few steps) likely reduces estimate quality for varying parameter ranges.
- **Parallel plan inefficiency** (exchange overhead/synchronization) became dominant as work per execution increased.

4) Missing evidence

- No Query Store runtime intervals for SQL machine date `2026-10-28` (available window is `2026-10-27 14:47–14:56` local).
- Actual execution plan runtime operator stats (spills, per-operator actual rows, exchange skew) were not captured here.
- Concurrent workload/session-level pressure during degraded minutes (blocking chains, scheduler pressure) not yet correlated.

5) Diagnosis

The performance incident is primarily a **Query Store-confirmed degradation of `query_id = 1`**, dominated by **`plan_id = 14`**.
Degradation occurred **while keeping the same plan**, with sharp growth in CPU and logical reads per execution.
Evidence points to a **parallelism-heavy, low-selectivity scan/aggregation pattern** on `dbo.Notes` with **highly skewed data distribution** on `ID/NAME`, making this query sensitive to runtime conditions and parameter ranges.

6) Recommended actions (read-only / safe)

1. Capture and compare **actual execution plans** for good vs bad intervals (focus on exchanges, scans, memory grants, spills).
2. Pull interval-level concurrency diagnostics for the same time window (`waits`, blocking, runnable tasks) to validate parallelism/lock amplification.
3. Review histogram detail for `ID`/`NAME` and estimate quality against observed row counts.
4. Evaluate whether this query should be rewritten to reduce scanned rows before aggregation.
5. If you want, I can provide a **single read-only diagnostic script** that correlates Query Store runtime, waits, and interval concurrency side-by-side for this query.

--- long running prompt --- not enough time to wait until it is completed

PROMPT 2 – PROPOSE POSSIBLE ACTIONS TO IMPROVE THE PERFORMANCE OF THE TOP QUERY

```
Investigate the SQL performance problem further based on analysis performed in PROMPT 1.

Analyze further on the already identified query id to understand if there are some possible actions (create index, update
statistics, fix execution plan, etc.) that could improve performances of that statement.

Do not ask me to provide Query IDs or Plan IDs if they can be discovered automatically.

Use only read-only diagnostic queries.

Do not create or drop indexes, update or drop statistics, force plans, clear caches, modify Query Store, or modify data.

Popose actions as list of items, including for each one the T-SQL statement (if any), the possible positive improvement and
possible side effects.

The proposed actions shout be sorted by most expected impact first
```

OUTPUT:

Based on the further diagnostics on the already identified top query (`query_id = 1, [dbo].[GiveNotes]`), here are the proposed actions, ordered by **expected impact**.

1. **Create a covering nonclustered index for the predicate/output pattern**

```
CREATE NONCLUSTERED INDEX IX_Notes_ID_INCLUDE_NAME
ON dbo.Notes (ID)
INCLUDE (NAME)
WITH (ONLINE = ON, SORT_IN_TEMPDB = ON);
```

- **Possible positive improvement**
  - Highest expected impact.
  - Reduces logical reads and CPU for `ID BETWEEN @N2 AND @N1`.
  - Should reduce parallelism pressure by reducing scanned pages.
- **Possible side effects**
  - Extra storage.
  - Higher write cost on `INSERT/UPDATE/DELETE`.
  - Build time and transaction log growth during creation.

2. **Mitigate parameter sensitivity in `dbo.GiveNotes` (test `OPTION (RECOMPILE)`)**

```
ALTER PROCEDURE dbo.GiveNotes
    @N1 INT = NULL,
    @N2 INT = NULL
AS
BEGIN
    SET NOCOUNT ON;

    SELECT COUNT(NAME), NAME, 'easyfilter' AS easyfilter
    FROM dbo.Notes
    WHERE ID BETWEEN @N2 AND @N1
    GROUP BY NAME
    OPTION (RECOMPILE);
END
GO
```

- **Possible positive improvement**
  - Better plan quality per execution when `@N1/@N2` ranges vary widely.
  - Can prevent reusing a bad sniffed plan.

- **Possible side effects**
  - More compile CPU per execution.
  - Can be expensive at very high call rates.

3. **Alternative to full recompilation: stabilize with `OPTIMIZE FOR UNKNOWN`**

```
ALTER PROCEDURE dbo.GiveNotes
    @N1 INT = NULL,
    @N2 INT = NULL
AS
BEGIN
    SET NOCOUNT ON;

    SELECT COUNT(NAME), NAME, 'easyfilter' AS easyfilter
    FROM dbo.Notes
    WHERE ID BETWEEN @N2 AND @N1
    GROUP BY NAME
    OPTION (OPTIMIZE FOR (@N1 UNKNOWN, @N2 UNKNOWN));
END
GO
```

- **Possible positive improvement**
  - More stable “middle-ground” plan than sniffed extremes.
  - Lower compile overhead than `RECOMPILE`.
- **Possible side effects**
  - May be suboptimal for both very small and very large ranges.

4. **If validated, force a better historical plan via Query Store**

```
EXEC sys.sp_query_store_force_plan @query_id = 1, @plan_id = 11;
-- or another validated better plan for your workload
```

- **Possible positive improvement**
  - Immediate regression control when a clearly better plan is known.
- **Possible side effects**
  - Forced plan can become suboptimal as data/workload changes.
  - Operational overhead to monitor and unforce if it regresses.
  - Requires Query Store write mode.

5. **Improve statistics quality on `dbo.Notes` key columns**

```
UPDATE STATISTICS dbo.Notes ([_WA_Sys_00000001_7D439ABD]) WITH FULLSCAN;
UPDATE STATISTICS dbo.Notes ([_WA_Sys_00000002_7D439ABD]) WITH FULLSCAN;
```

- **Possible positive improvement**
  - Better cardinality estimates on skewed data distribution (`ID`, `NAME`), potentially improving plan choice.
- **Possible side effects**
  - CPU/IO overhead during update.
  - May trigger recompiles and plan changes.

Why this order

- Your evidence shows very high read/CPU amplification and missing-index signals on `ID` + `NAME`; that makes indexing the top expected gain.
- Parameter sensitivity is strongly suggested by major plan/perf variance and different compiled parameter contexts.
- Plan forcing and stats changes are useful controls but typically secondary to fixing access path and sniffing behavior.